

A AURORA ENTREGA O COMANDO À
INTELIGÊNCIA ARTIFICIAL

1. Otimização

das
interações
que aperfeiçoa
o comando
autônomo

LLM

Large Language Model

LISTA DE FIGURAS

Figura 1 – Fluxo de um sistema com RAG	6
Figura 2 – Fluxo de predição de um LLM	8
Figura 3 – LLMs podem facilmente alucinar nas respostas	9
Figura 4 – Funcionamento do RAG	13
Figura 5 – <i>Embeddings</i> e <i>vector databases</i>	14
Figura 6 – Componentes de um RAG	17
Figura 7 – Similaridade do cosseno entre 2 vetores de <i>embeddings</i>	19
Figura 8 – Interpretação da similaridade do cosseno entre 2 vetores de <i>embeddings</i>	19
Figura 9 – Representação de textos como <i>embeddings</i>	23
Figura 10 – Busca semântica	24
Figura 11 Visualizando a distância entre embeddings com 2 dimensões após uso de PCA	28

LISTA DE CÓDIGOS-FONTE

Código-fonte 1: Exemplo de busca semântica com <i>embeddings</i>	7
Código-fonte 2: Exemplo de resposta para pergunta simples, que não exige contexto	11
Código-fonte 3: Exemplo de resposta para pergunta que exige contexto atualizado	12
Código-fonte 4: Coleção de textos preparada para recuperação	16
Código-fonte 5: RAG: Passo a passo.....	22
Código-fonte 6: Compreendendo o processo de busca semântica	27
Código-fonte 7: Sistema RAG para responder a dúvidas sobre o processo seletivo da FIAP	33

SUMÁRIO

RETRIEVAL AUGMENTED GENERATION	5
1 A NOVA GERAÇÃO DE SISTEMAS COM IA	5
2 MODELOS DE LINGUAGEM E SUAS LIMITAÇÕES	8
3 INTRODUÇÃO AO RAG	12
4 COMPONENTES PRINCIPAIS DO RAG	16
4.1 Fonte de conhecimento e pré-processamento de texto	17
4.2 Banco de dados vetorial (<i>Vector Database</i>)	18
4.3 <i>Retriever</i> (mecanismo de recuperação)	18
4.4 <i>Generator</i> (modelo gerador)	20
4.5 Integração e coordenação dos componentes	20
4.6 Exemplo prático simplificado em Python	20
5 <i>EMBEDDINGS</i> PARA RECUPERAÇÃO	23
6 APLICAÇÕES DO RAG	29
7 DESAFIOS	34
8 CONCLUSÃO	37
REFERÊNCIAS	39

RETRIEVAL AUGMENTED GENERATION

1 A NOVA GERAÇÃO DE SISTEMAS COM IA

As aplicações de inteligência artificial baseadas em linguagem natural evoluíram rapidamente nos últimos anos. Começamos com mecanismos de busca tradicionais que varriam grandes volumes de texto e retornavam documentos relevantes com base em palavras-chave. Essa abordagem foi posteriormente complementada por sistemas que entendiam consultas de linguagem natural, mas ainda assim dependiam de regras explícitas ou de índices pré-construídos. A chegada dos modelos de linguagem de grande porte (LLMs), como GPT-4, BERT e seus derivados, representou um avanço significativo, permitindo que computadores gerassem textos coerentes com alto grau de fluência, muitas vezes indistinguível do texto humano.

Apesar dessa evolução, os LLMs apresentam limitações fundamentais. Eles operam essencialmente como máquinas de predição de sequência de tokens treinadas em grandes *corpora* históricos de texto, e, portanto, seu conhecimento é estático e limitado aos dados em que foram treinados. Mesmo modelos amplamente usados podem gerar respostas imprecisas, desatualizadas ou até inventadas, fenômeno conhecido como alucinação, quando questionados sobre fatos específicos ou informações que não estavam presentes em seus dados de treinamento. Além disso, eles não conseguem acessar diretamente bases de dados atualizadas ou conteúdos proprietários, o que limita sua utilidade em aplicações empresariais ou acadêmicas, em que a precisão e evidência são essenciais, como perguntas baseadas em documentação interna ou artigos científicos publicados depois do corte de treinamento do modelo.

É neste contexto que surge o conceito de *Retrieval-Augmented Generation* (RAG), ou, em português, Geração Aumentada por Recuperação. RAG é uma técnica híbrida que combina os pontos fortes de mecanismos de recuperação de informação com as capacidades de geração dos LLMs. Em vez de produzir respostas com base unicamente na memória implícita do modelo, um sistema RAG recorre a uma base de conhecimento externa para “consultar” dados relevantes antes de gerar a resposta

final. Em termos simples, o modelo primeiro identifica trechos de texto ou documentos que podem responder à pergunta e só então utiliza essas evidências junto com a pergunta do usuário para criar uma resposta fundamentada. Essa abordagem permite respostas mais precisas, atualizadas e contextualizadas em domínios específicos ou em contextos personalizados de documentos.

Na arquitetura RAG, a etapa de recuperação de dados é tão crítica quanto a geração de texto. Sistemas eficazes de recuperação conseguem selecionar trechos de uma coleção de documentos que sejam semanticamente relevantes para a pergunta do usuário, permitindo que o modelo não dependa de sua representação pré-treinada para responder. Essa integração é claramente delineada em esforços acadêmicos e documentos técnicos sobre o tema. O *pipeline* típico de um sistema RAG consiste em indexar uma coleção de documentos com representações vetoriais (*embeddings*), procurar os vetores mais semelhantes à consulta do usuário e, finalmente, alimentar o modelo de geração com esses trechos selecionados como contexto adicional.

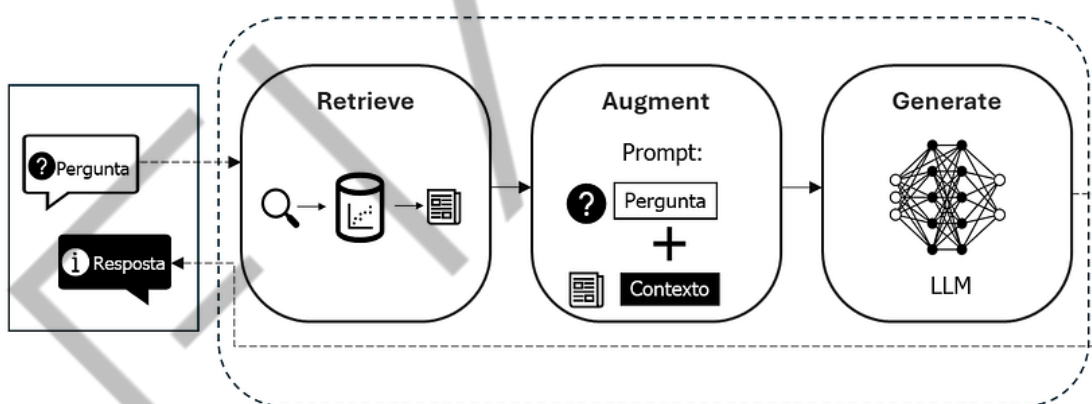


Figura 1 – Fluxo de um sistema com RAG
Fonte: viso.ai (2024)

Apesar de seu potencial, RAG não elimina todos os problemas dos LLMs nem substitui completamente mecanismos de busca ou bases de dados transacionais; ela representa uma abordagem integrada que amplia o alcance de modelos generativos para tarefas que exigem precisão factual e relevância contextual. Em aplicações como assistentes técnicos com acesso a manuais internos, agentes de suporte ao cliente que consultam FAQs corporativas ou sistemas de perguntas e respostas baseados em regulamentações legais ou científicas, o emprego de RAG tem demonstrado ganhos substanciais em qualidade de resposta.

Para ilustrar como um sistema RAG pode ser iniciado na prática utilizando Python, considere o exemplo simplificado abaixo, que carrega uma coleção de documentos, divide-os em blocos menores (*chunking*), gera *embeddings* para cada bloco e, por fim, armazena esses *embeddings* em uma estrutura de dados que permite a recuperação de trechos relevantes:

```
1 | # Exemplo mínimo de pipeline RAG com Python
2 | from sentence_transformers import SentenceTransformer
3 | from sklearn.metrics.pairwise import cosine_similarity
4 | import numpy as np
5 |
6 | # Carrega modelo de embeddings
7 | emb_model = SentenceTransformer("all-MiniLM-L6-v2")
8 |
9 | # Documentos de exemplo
10| docs = [
11|     "O Brasil tem 26 estados e 1 distrito federal.",
12|     "São Paulo é o estado mais populoso do Brasil.",
13|     "A capital do Brasil é Brasília."
14| ]
15|
16| # Gera embeddings
17| doc_embeddings = emb_model.encode(docs)
18|
19| # Exemplo de consulta
20| query = "Qual é a capital do Brasil?"
21| query_emb = emb_model.encode([query])
22|
23| # Recupera índice do documento mais similar
24| similarities = cosine_similarity(query_emb,
25| doc_embeddings)
26| best_idx = np.argmax(similarities)
27| print(f"Documento mais relevante: {docs[best_idx]}")
```

Código-fonte 1: Exemplo de busca semântica com *embeddings*
Fonte: Elaborado pelo autor (2026)

Esse simples exemplo ilustra os primeiros passos envolvidos em um *pipeline* RAG: representar texto e consultas no espaço vetorial semântico e identificar quais documentos ou trechos são mais relevantes para a consulta. Em sistemas reais, a etapa de geração subsequente envolveria passar o contexto recuperado para um LLM com instruções adequadas (prompt) para produzir uma resposta coerente e fundamentada.

2 MODELOS DE LINGUAGEM E SUAS LIMITAÇÕES

Os modelos de linguagem de grande porte representam uma das inovações mais transformadoras no campo da inteligência artificial nas últimas décadas. Eles são baseados em arquiteturas de redes neurais profundas chamadas *Transformers*, que aprendem a gerar linguagem natural fluente ao serem expostos a enormes *corpora* de texto durante a fase de treinamento. Essa arquitetura foi proposta inicialmente no artigo “*Attention Is All You Need*” e se tornou a base dos modelos atuais por sua capacidade de reconhecer padrões de dependência em sequências de texto com eficiência e escala inéditas.

LLMs como GPT, Claude e LLaMA são treinados em grandes quantidades de dados textuais que incluem artigos, livros, sites e códigos. Eles operam como máquinas de previsão de tokens, ou seja, aprendem a prever a próxima palavra em um contexto dado, repetindo esse processo até formar frases e parágrafos coerentes. Esse processo de inferência, realizado token a token, permite que os modelos realizem tarefas como responder perguntas, traduzir idiomas, resumir textos e até gerar código.

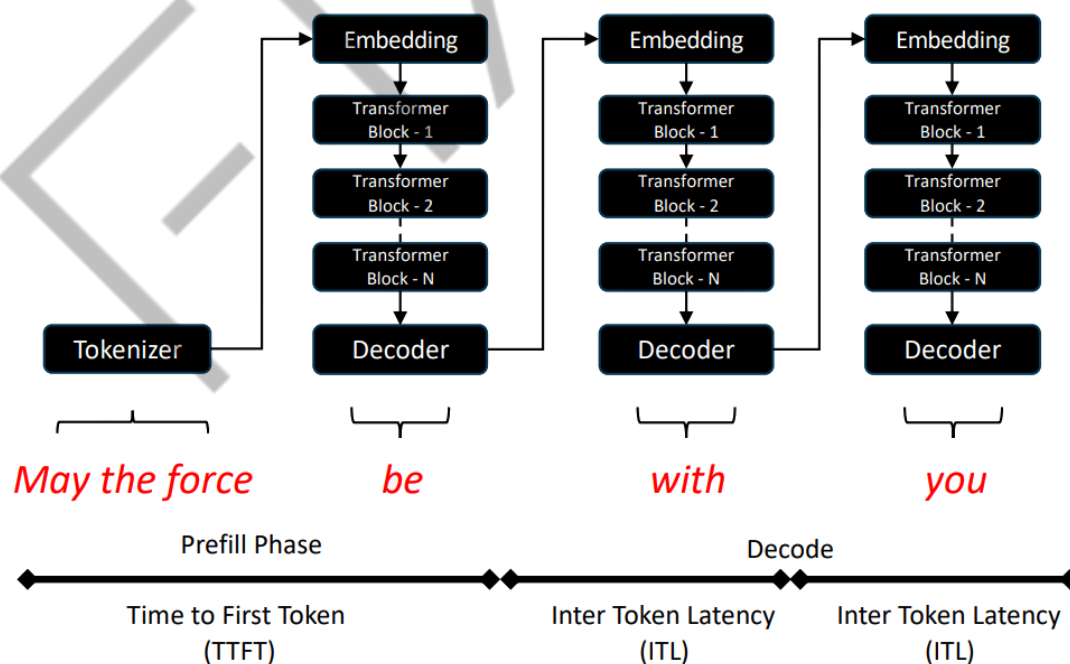


Figura 2 – Fluxo de previsão de um LLM
Fonte: viso.ai (2025)

Apesar de sua versatilidade, os LLMs apresentam limitações fundamentais decorrentes de sua natureza probabilística e da forma como são treinados. Uma das limitações mais discutidas é o fenômeno conhecido como “alucinação” (*hallucination*), que ocorrem quando o modelo produz respostas que são sintaticamente plausíveis e coerentes em termos de linguagem, mas que não correspondem a fatos reais ou não podem ser verificadas em uma fonte confiável. Isso acontece porque os LLMs não possuem um mecanismo de verificação de factualidade interno; eles simplesmente extrapolam a partir dos padrões que aprenderam durante o treinamento, o que pode resultar em informações inventadas ou incorretas.

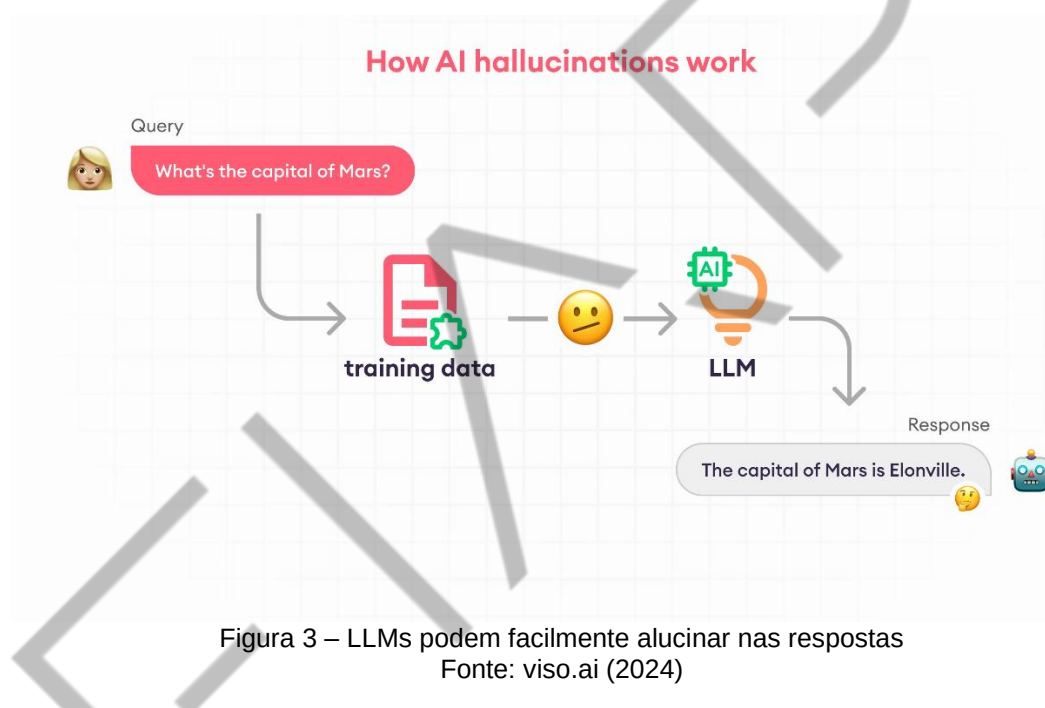


Figura 3 – LLMs podem facilmente alucinar nas respostas
Fonte: viso.ai (2024)

Uma causa recorrente desse fenômeno é o próprio objetivo de treinamento dos LLMs: maximizar a probabilidade de sequência de texto observada durante a fase de pré-treinamento. Esse processo não ensina o modelo a distinguir entre fatos corretos e falsas generalizações, mas sim a otimizar a plausibilidade estatística da sequência de palavras. Estudiosos argumentam que essa tendência não é apenas uma falha ocasional, mas uma característica inerente à forma como os modelos são construídos e avaliados, muitas vezes incentivando o modelo a “adivinhar” respostas plausíveis mesmo quando não possui dados suficientes para ser factual.

Além das alucinações, os LLMs enfrentam outras limitações importantes. Uma delas é o *context window*, ou “janela de contexto”, que define quanto de informação o

modelo pode considerar simultaneamente ao gerar uma resposta. Essa janela é limitada e, quando a entrada do usuário ou a conversa ultrapassa esse limite, o modelo perde acesso às partes iniciais do contexto, o que pode resultar em respostas incoerentes ou desconexas em diálogos longos.

Outra limitação significativa é o chamado *knowledge cut-off*, ou limite de conhecimento. Como os LLMs são treinados uma única vez em um conjunto de dados estático, eles não conseguem incorporar automaticamente informações posteriores ao corte de treinamento. Isso significa que um modelo pode não saber de eventos recentes ou avanços científicos que ocorreram após esse ponto, o que limita severamente sua utilidade em aplicações que exigem conhecimento atualizado.

Os modelos também enfrentam desafios relacionados à memória de longo prazo. Eles não mantêm um entendimento persistente ou uma representação duradoura de fatos ou experiências específicas fora do escopo imediato da inferência. Em outras palavras, eles não têm uma forma de “lembrar” permanentemente de fatos adquiridos durante conversas passadas ou de promover aprendizado contínuo ao longo do tempo, o que pode ser crítico em sistemas interativos de assistência ou aprendizado personalizado.

A maneira como os LLMs são treinados também pode introduzir vieses. Esses modelos refletem e, em alguns casos, amplificam preconceitos presentes em seus dados de treinamento, reproduzindo estereótipos ou respostas tendenciosas em contextos sensíveis. Esses vieses não são apenas éticos; podem impactar a confiabilidade e a segurança das aplicações em setores jurídicos, financeiros ou de saúde.

Para ilustrar uma dessas limitações na prática, considere o seguinte exemplo em Python usando um LLM por meio da API da OpenAI. Aqui, um usuário faz uma pergunta que pode levar a uma alucinação:

```
1 | from openai import OpenAI
2 | from google.colab import userdata
3 | import os
4 |
5 | os.environ['OPENAI_API_KEY'] =
  userdata.get('OPENAI_API_KEY')
6 |
7 | # Crie o cliente com sua chave
```

```
8 | client = OpenAI()
9 |
10| pergunta = "Qual é a capital da França?"
11|
12| # Chamada ao modelo (ChatGPT)
13| response = client.chat.completions.create(
14|     model="gpt-4o-mini", # modelo leve e econômico
15|     messages=[
16|         {"role": "system", "content": "Você é um assistente útil."},
17|         {"role": "user", "content": pergunta}
18|     ],
19|     max_tokens=100,
20|     temperature=0.7
21| )
22|
23| print("Resposta do LLM:",
response.choices[0].message.content)
```

Código-fonte 2: Exemplo de resposta para pergunta simples, que não exige contexto
Fonte: Elaborado pelo autor (2026)

Esse código simples provavelmente retornará corretamente “Paris” porque se trata de um fato bem estabelecido. No entanto, se a pergunta for mais específica ou requerer conhecimento atualizado sobre um evento recente não presente no treinamento do modelo, a resposta pode ser incorreta ou inventada. Analise:

```
1 | from openai import OpenAI
2 | from google.colab import userdata
3 | import os
4 |
5 | os.environ['OPENAI_API_KEY'] =
userdata.get('OPENAI_API_KEY')
6 |
7 | # Crie o cliente com sua chave
8 | client = OpenAI()
9 |
10| pergunta = "Quem é o melhor jogador do mundo? Responda apenas o nome do jogador"
11|
12| # Chamada ao modelo (ChatGPT)
13| response = client.chat.completions.create(
14|     model="gpt-4o-mini", # modelo leve e econômico
15|     messages=[
16|
17|         {"role": "user", "content": pergunta}
18|     ],
19|     max_tokens=100,
```

```
20 |     temperature=0
21 | )
22 |
23 | print("Resposta do LLM:",
    response.choices[0].message.content)
```

Código-fonte 3: Exemplo de resposta para pergunta que exige contexto atualizado
Fonte: Elaborado pelo autor (2026)

Nesse caso, se o modelo não tiver acesso a informações atualizadas, ele pode produzir uma resposta fantasiosa ou incorreta devido ao seu *knowledge cut-off*, o que ilustra a limitação prática de depender apenas de LLMs sem mecanismos de atualização ou consulta a bases externas de conhecimento.

Essas limitações tornam evidente que, embora os LLMs sejam ferramentas poderosas para geração natural de linguagem, eles não são suficientes por si só para aplicações que exigem precisão factual, consistência ao longo de contextos longos ou integração com bases de conhecimento proprietárias. Esse conjunto de restrições fundamenta a necessidade de arquiteturas como *Retrieval-Augmented Generation*, que buscam endereçar especificamente esses desafios ao combinar recuperação de informação externa com a geração de texto dos LLMs.

3 INTRODUÇÃO AO RAG

A técnica conhecida como *Retrieval-Augmented Generation* representa um avanço importante no campo de sistemas de geração de linguagem natural, pois combina de forma explícita dois paradigmas distintos de processamento de linguagem: a recuperação de informação e a geração de texto por grandes modelos de linguagem. Em vez de depender apenas do conhecimento aprendido durante o treinamento, um sistema RAG primeiro busca contextos relevantes em uma base de dados externa, que pode incluir documentos, bancos de dados ou coleções de textos especializados, e depois usa essas informações para fundamentar e enriquecer a resposta gerada pelo modelo. Essa abordagem procura melhorar limitações fundamentais dos LLMs puros.

Quando representamos o fluxo de uma aplicação tradicional de LLM sem RAG, a sequência de processamento consiste em receber a entrada do usuário, processá-la internamente com base nos padrões aprendidos e gerar uma resposta diretamente.

Com RAG, a sequência inclui uma etapa adicional anterior à geração, em que a consulta do usuário é usada para recuperar trechos de texto relevantes de uma base de conhecimento previamente indexada e vetorizada. Esses trechos são então acrescentados ao prompt enviado ao LLM, que os utiliza como contexto adicional ao produzir a resposta final. O resultado é uma resposta mais precisa, evidenciada por fontes reais e muitas vezes atualizada em relação ao corte de treinamento do modelo.

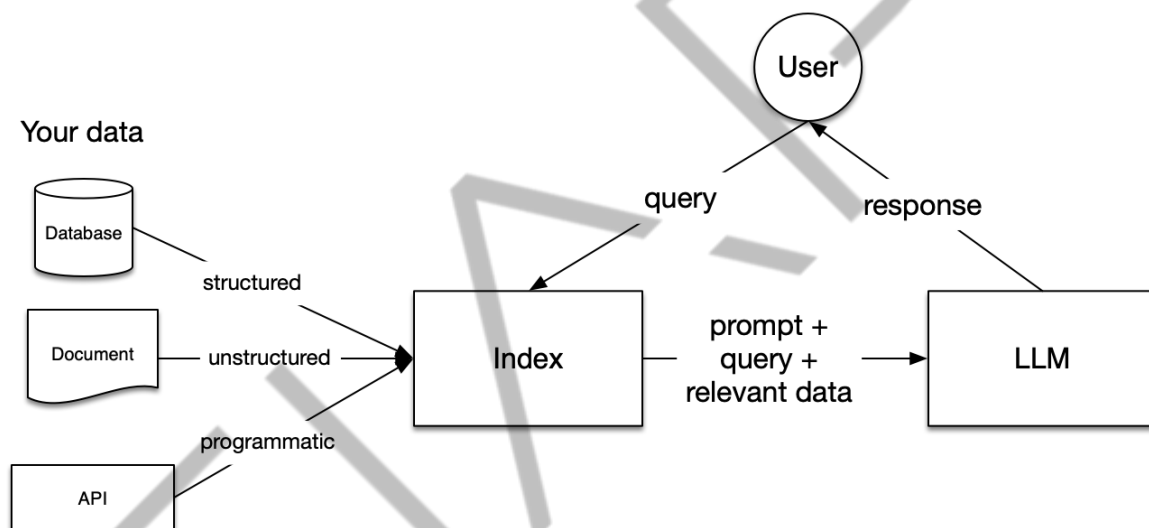


Figura 4 – Funcionamento do RAG
Fonte: viso.ai (2025)

A literatura técnica descreve o RAG como uma estrutura híbrida de IA que otimiza a saída de um modelo ligando-o a uma base de conhecimento externa antes de gerar a resposta. Essa base de conhecimento pode ser atualizada continuamente, permitindo que o sistema responda a perguntas sobre fatos recentes ou informações proprietárias de uma organização sem a necessidade de retreinar o LLM. A técnica surgiu formalmente em trabalhos de pesquisa acadêmica dedicados a tarefas de processamento de linguagem natural intensivas em conhecimento (*Knowledge-Intensive NLP Tasks*), mas rapidamente foi adotada em desenvolvimento prático por empresas que buscam respostas mais confiáveis e contextualizadas de seus agentes de IA.

Um aspecto fundamental do RAG é o uso de representações vetoriais semânticas (*embeddings*) para efetuar a recuperação de trechos de texto relevantes no momento da consulta. Nessa etapa, tanto o texto dos documentos quanto a consulta do usuário são transformados em vetores em um espaço de alta dimensão, de modo que a similaridade entre esses vetores possa ser utilizada para identificar partes de documentos cuja semântica corresponda mais de perto à intenção da pergunta. Esse processo de indexação e busca vetorial é frequentemente implementado através de bancos de dados vetoriais (*vector databases*), que permitem consultas rápidas e eficientes sobre grandes coleções de dados.

Além disso, o RAG pode reduzir a necessidade de custos computacionais associados ao retreinamento constante dos LLMs. Tradicionalmente, atualizar um LLM com novos dados exige uma nova fase de treinamento, um processo que é caro e demorado. Ao invés disso, o RAG apenas atualiza a base de conhecimento externa e o índice vetorial, deixando o modelo de geração inalterado, o que constitui uma abordagem mais prática e econômica para cenários em que a atualização contínua de dados é crítica.

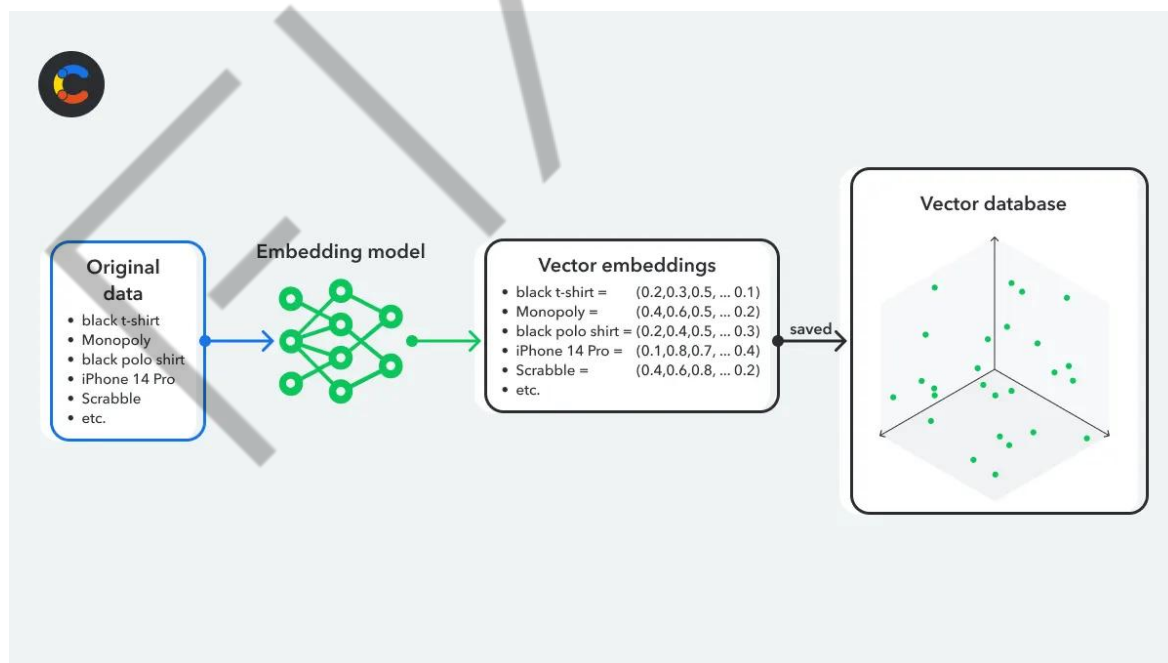


Figura 5 – *Embeddings* e *vector databases*
Fonte: viso.ai (2024)

Do ponto de vista conceitual, um sistema RAG pode ser dividido em duas partes principais: o *retriever* (mecanismo de recuperação) e o *generator* (modelo de

geração). O *retriever* é responsável por buscar as partes mais relevantes da base de conhecimento com base na consulta do usuário. Depois, o *generator*, normalmente um LLM capaz de lidar com grandes quantidades de texto no prompt, recebe tanto a consulta quanto os trechos de documentos recuperados e produz uma resposta que combina compreensão linguística com evidências contextuais.

Uma vantagem prática dessa estrutura é que ela permite que desenvolvedores construam aplicações específicas de perguntas e respostas (Q&A) sobre conjuntos de dados proprietários ou técnicos sem precisar expor esses dados ao próprio treinamento do modelo. Isso se aplica, por exemplo, a sistemas que respondem a perguntas sobre manuais de produtos, documentação legal ou dados internos de uma organização, em que a precisão e a atualidade da resposta são essenciais.

Para exemplificar de forma prática como uma etapa de recuperação pode ser integrada a um fluxo em Python, considere o uso de uma biblioteca que suporte indexação vetorial, como ChromaDB ou FAISS, junto com LangChain, que é uma biblioteca que facilita a construção de *pipelines* RAG. Embora detalhes completos de indexação e geração estejam além do escopo desse material, o trecho de código abaixo ilustra como uma coleção de textos pode ser preparada para recuperação:

```
1 | !pip install langchain openai chromadb
   langchain_community langchain_openai
2 |
3 | from langchain_community.vectorstores import Chroma
4 | from langchain_openai import OpenAIEmbeddings
5 | import os
6 | from google.colab import userdata
7 |
8 | os.environ['OPENAI_API_KEY'] =
   userdata.get('OPENAI_API_KEY')
9 |
10| # Criar embeddings e indexar textos
11| embeddings = OpenAIEmbeddings()
12| textos = ["Documentação de produto A", "Manual de
   operação B", "Relatório técnico C"]
13| vector_db = Chroma.from_texts(texts=textos,
   embedding=embeddings)
14|
15| # Consulta de exemplos
16| consulta = "Como configurar o produto A?"
17| resultados = vector_db.similarity_search(consulta, k=1)
18| for r in resultados:
19|     print("Trecho relevante:", r.page_content)
```

Código-fonte 4: Coleção de textos preparada para recuperação
Fonte: Elaborado pelo autor (2026)

Nesse exemplo, um conjunto de textos é convertido em vetores com base em um modelo de *embeddings* e armazenado em uma base vetorial. Quando uma consulta é submetida, os trechos de texto mais semanticamente próximos são retornados, prontos para serem usados como contexto em uma geração posterior com um LLM. A técnica permite separar claramente a recuperação de informações da geração de linguagem, facilitando a modularidade e a interpretação de resultados.

Em síntese, o RAG representa um desenvolvimento importante na construção de sistemas de IA generativa mais precisos, contextualizados e adaptáveis a domínios específicos, superando algumas limitações dos LLMs tradicionais ao fornecer respostas mais fundamentadas em dados externos confiáveis.

4 COMPONENTES PRINCIPAIS DO RAG

Aqui vamos aprofundar os componentes fundamentais da arquitetura RAG, descrevendo com clareza quais são os blocos que compõem um sistema, como eles se interconectam e qual é o papel de cada um no fluxo de trabalho. A compreensão detalhada desses elementos é essencial para projetar, implementar e otimizar sistemas reais de IA com recuperação de conhecimento externo integrada.

Um sistema RAG é tipicamente composto por módulos que operam de maneira colaborativa: um mecanismo de *retrieval*, que busca trechos de texto relevantes em uma base de conhecimento, um sistema de indexação, que armazena representações semânticas dos dados, e um gerador, normalmente um grande modelo de linguagem, que cria respostas informadas com base no contexto recuperado. A combinação desses módulos permite que se obtenham respostas que vão além dos limites dos dados usados no treinamento do modelo, fornecendo informação atualizada, específica e ancorada em fontes verificáveis.

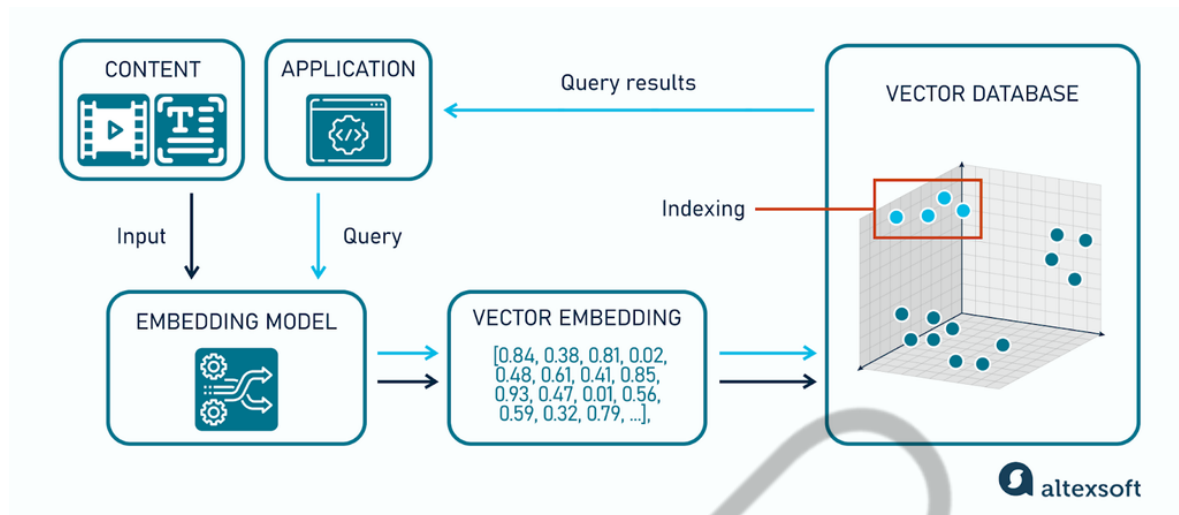


Figura 6 – Componentes de um RAG
Fonte: viso.ai (2025)

4.1 Fonte de conhecimento e pré-processamento de texto

O ponto de partida em qualquer sistema RAG é a base de conhecimento externa, que pode incluir documentos em vários formatos como textos livres, artigos científicos, planilhas, PDFs ou qualquer outro material que contenha informação relevante para o domínio de interesse. Antes de usar esses textos no sistema, é necessário realizar uma etapa de pré-processamento em que os documentos são limpos, normalizados e divididos em partes menores (técnica conhecida como *chunking*). Esse processo de fragmentação é importante porque modelos de linguagem e sistemas de recuperação operam com maior eficiência quando lidam com trechos de texto de tamanho moderado em vez de blocos enormes de dados.

Após a divisão dos textos, a próxima etapa consiste em gerar representações vetoriais (*embeddings*) que capturem o significado semântico de cada trecho. Representações vetoriais são vetores numéricos de alta dimensão que codificam as características semânticas de uma sentença ou parágrafo de forma que conteúdos semanticamente semelhantes fiquem próximos no espaço vetorial. Esses *embeddings* são gerados por modelos pré-treinados de aprendizado profundo, como modelos de *sentence embeddings* baseados em *Transformers*.

4.2 Banco de dados vetorial (*Vector Database*)

Uma vez que os *embeddings* são gerados, eles precisam ser armazenados de forma eficiente para suportar operações de busca rápida baseada em similaridade.

Para isso, utilizam-se os chamados bancos de dados vetoriais, que são sistemas de armazenamento especializados capazes de indexar e consultar vetores de forma eficiente. Ao contrário de bancos de dados tradicionais que buscam correspondência por chaves ou consultas SQL, os bancos vetoriais permitem recuperar os vetores mais semelhantes ao vetor de consulta usando técnicas de similaridade semântica. Isso é essencial para encontrar os trechos de texto que possuem conteúdo conceitual próximo ao significado da pergunta do usuário.

A escolha de um bom banco de dados vetorial impacta diretamente na performance e na qualidade do sistema RAG, pois ele deve ser capaz de lidar com dezenas de milhares ou milhões de vetores, mantendo consultas rápidas e precisas.

Exemplos populares de tecnologias nessa categoria incluem FAISS, Milvus, ChromaDB e outros que implementam algoritmos de busca aproximada de vizinhos para escalar a recuperação sem sacrificar a velocidade.

4.3 *Retriever* (mecanismo de recuperação)

O componente conhecido como *retriever* é o responsável por identificar, entre todos os vetores indexados, aqueles que são mais relevantes para a consulta feita pelo usuário. Essa etapa é feita tipicamente transformando a consulta em seu próprio vetor de *embeddings* e, em seguida, comparando-o com os vetores armazenados no banco vetorial para recuperar os itens semanticamente mais próximos. Essa comparação pode ser feita usando métricas como similaridade de cosseno ou distância euclidiana em espaços de alta dimensão.

Cosine Similarity

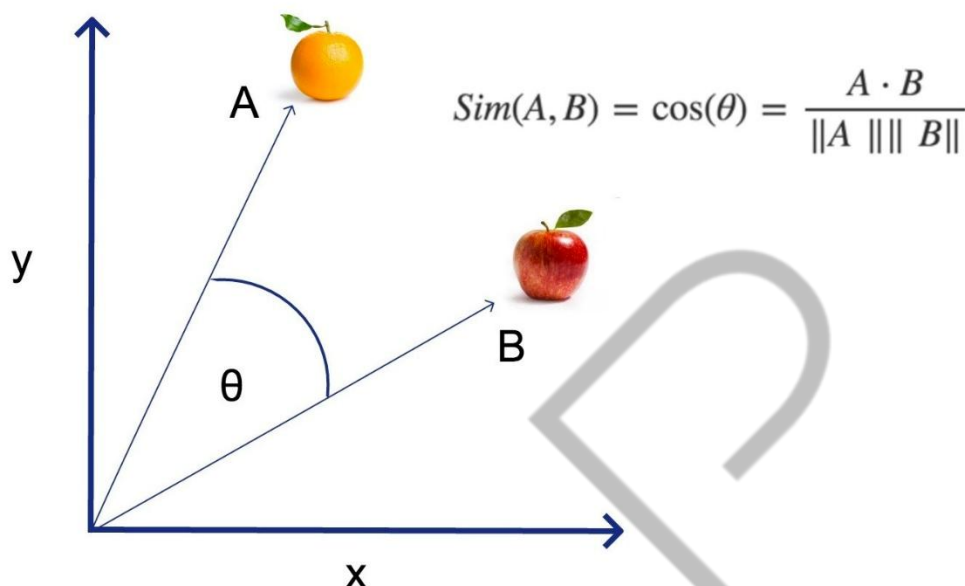


Figura 7 – Similaridade do cosseno entre 2 vetores de *embeddings*
Fonte: viso.ai (2025)

O objetivo dessa comparação é retornar um conjunto de trechos de texto que contém a informação necessária para responder à pergunta do usuário de forma mais provável. Em implementações práticas, costuma-se recuperar os K vetores mais próximos, por exemplo, os 5 ou 10 trechos de documentos que mais se alinham semanticamente à consulta, para maximizar a probabilidade de abranger contexto relevante enquanto a eficiência computacional se mantém.

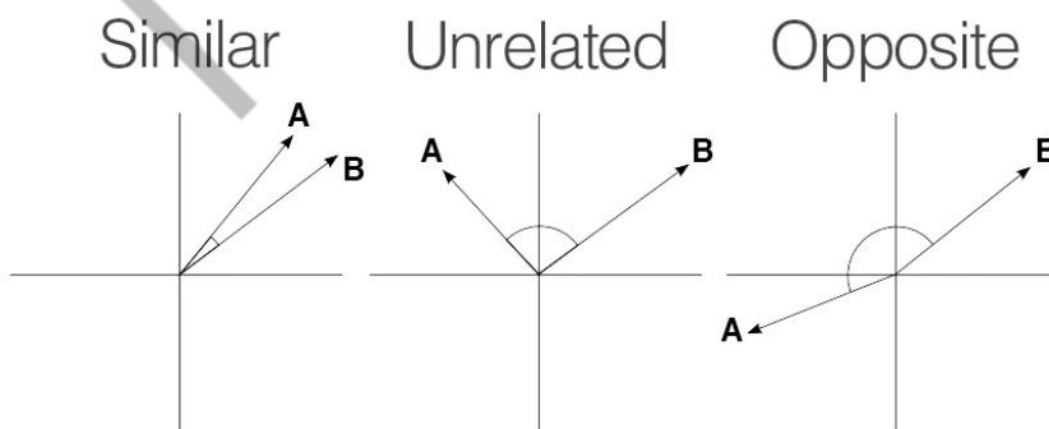


Figura 8 – Interpretação da similaridade do cosseno entre 2 vetores de *embeddings*
Fonte: viso.ai (2025)

4.4 Generator (modelo gerador)

Depois que o mecanismo de *retrieval* identifica e recupera os trechos mais relevantes da base de conhecimento, essa informação é passada como contexto adicional para o modelo gerador, que geralmente é um grande modelo de linguagem. Esse modelo recebe tanto a pergunta original quanto os trechos recuperados como parte do prompt de entrada. O uso desses trechos como contexto permite que o modelo produza uma resposta que não é só formulada em linguagem natural fluente, mas também está ancorada em evidências extraídas da base de dados.

Nessa etapa, o modelo pode ser configurado para estruturar a resposta de diferentes maneiras, como fornecer citações diretas das fontes, sumarizar a informação recuperada ou combinar vários trechos para compor uma resposta coerente e contextualizada ao usuário. A eficácia do modelo gerador depende tanto da qualidade dos trechos recuperados quanto da maneira como o contexto é incorporado no prompt, levando em conta limites de tamanho e estratégias de *Prompt Engineering*.

4.5 Integração e coordenação dos componentes

O papel crucial de todo sistema RAG é coordenar de forma eficiente a interação entre o *retriever*, o banco de dados vetorial e o modelo gerador. O fluxo de dados típico começa quando o usuário envia uma pergunta, que é convertida em um vetor de *embeddings*. Esse vetor é consultado no banco vetorial para localizar trechos relevantes e os trechos recuperados são então agregados ao prompt do modelo *generator*, que produz a resposta final. Esse processo conjuga recuperação e geração de forma integrada, trazendo para a resposta elementos factuais atualizados e relevantes que não estavam no treinamento original do LLM.

4.6 Exemplo prático simplificado em Python

A seguir, temos um exemplo simplificado que ilustra a criação de um banco de dados vetorial e a recuperação de trechos com uma biblioteca como FAISS e um modelo de *sentence embeddings*. Observe que essa é uma versão didática e que

implementações reais incluem etapas adicionais de pré-processamento e integração com modelos geradores:

```
1 | !pip install faiss-cpu sentence-transformers openai
2 |
3 |
4 |
5 | # 1) IMPORTS
6 | from sentence_transformers import SentenceTransformer
7 | import faiss
8 | import numpy as np
9 | from openai import OpenAI
10| import os
11| from google.colab import userdata
12|
13| os.environ['OPENAI_API_KEY'] =
userdata.get('OPENAI_API_KEY')
14|
15| # 2) EMBEDDING MODEL
16|
17| modelo = SentenceTransformer("all-MiniLM-L6-v2")
18|
19| # Base de conhecimento
20| textos = [
21|     "Manual de usuário: Para instalar o produto, execute
o instalador e siga as instruções na tela.",
22|     "Guia de instalação: O produto requer Python 3.10 ou
superior.",
23|     "Especificações técnicas: O sistema suporta Windows
e Linux."
24| ]
25|
26| embeddings = modelo.encode(textos)
27|
28| # 3) FAISS INDEX (VectorDB)
29| dim = embeddings.shape[1]
30| index = faiss.IndexFlatL2(dim)
31| index.add(np.array(embeddings))
32|
33| # 4) CONSULTA (retrieval)
34| consulta = "Como instalar o produto?"
35| consulta_emb = modelo.encode([consulta])
36|
37| distancias, indices =
index.search(np.array(consulta_emb), k=2)
38|
39| contextos_recuperados = [textos[i] for i in indices[0]]
40|
41| print("Trechos recuperados:")
42| for trecho in contextos_recuperados:
```

```
43|     print("-", trecho)
44|
45| # 5) OPENAI GENERATOR (GPT-4o-mini)
46| client = OpenAI()
47|
48| # Construção do prompt RAG
49| prompt = f"""
50| Use exclusivamente o contexto abaixo para responder à
51| pergunta.
52| Contexto:
53| {chr(10).join(contextos_recuperados)}
54|
55| Pergunta:
56| {consulta}
57| """
58|
59| response = client.responses.create(
60|     model="gpt-4o-mini",
61|     input=prompt,
62|     max_output_tokens=200,
63|     temperature=0.2
64| )
65|
66| print("\nResposta final do modelo:")
67| print(response.output_text)
```

Código-fonte 5: RAG: Passo a passo
Fonte: Elaborado pelo autor (2026)

Esse código demonstra, de forma prática, como um vetor de consulta é comparado com vetores previamente indexados para recuperar trechos semanticamente similares. Em um sistema RAG completo, o último passo é enviar esses trechos recuperados ao gerador (por exemplo, um GPT-4 via API) para produzir uma resposta fundamentada e contextualizada.

Aqui apresentamos os componentes principais que sustentam um sistema RAG: a base de conhecimento pré-processada, o banco de dados vetorial, o mecanismo de recuperação e o modelo gerador, juntamente com a coordenação desses elementos em um *pipeline* coerente. Entender cada um desses blocos é fundamental para projetar soluções robustas de IA capazes de fornecer respostas mais precisas, atuais e confiáveis do que um modelo de linguagem operando isoladamente.

5 EMBEDDINGS PARA RECUPERAÇÃO

Aqui vamos explicar com profundidade o conceito de *embeddings* semânticos e sua função central na etapa de recuperação (*retrieval*) de um sistema RAG. Entender como *embeddings* funcionam e como eles são utilizados em tarefas de busca semântica e recuperação de informação é essencial para projetar sistemas eficazes.

Quando lidamos com textos em linguagem natural, computadores não conseguem interpretar significado diretamente a partir de palavras como nós, seres humanos, fazemos. Para tornar o texto compreensível por algoritmos, é necessário convertê-lo em uma forma numérica que preserve relações de significado entre diferentes partes do texto. Essa representação numérica é chamada de *embedding*, um vetor denso de números que captura características semânticas do texto ou de outros tipos de dados. Eles permitem que palavras, frases ou documentos sejam representados como vetores em um espaço vetorial no qual itens semanticamente semelhantes ficam próximos uns dos outros. Essa representação vetorial fornece a base matemática para comparar a similaridade entre diferentes textos de forma eficiente.

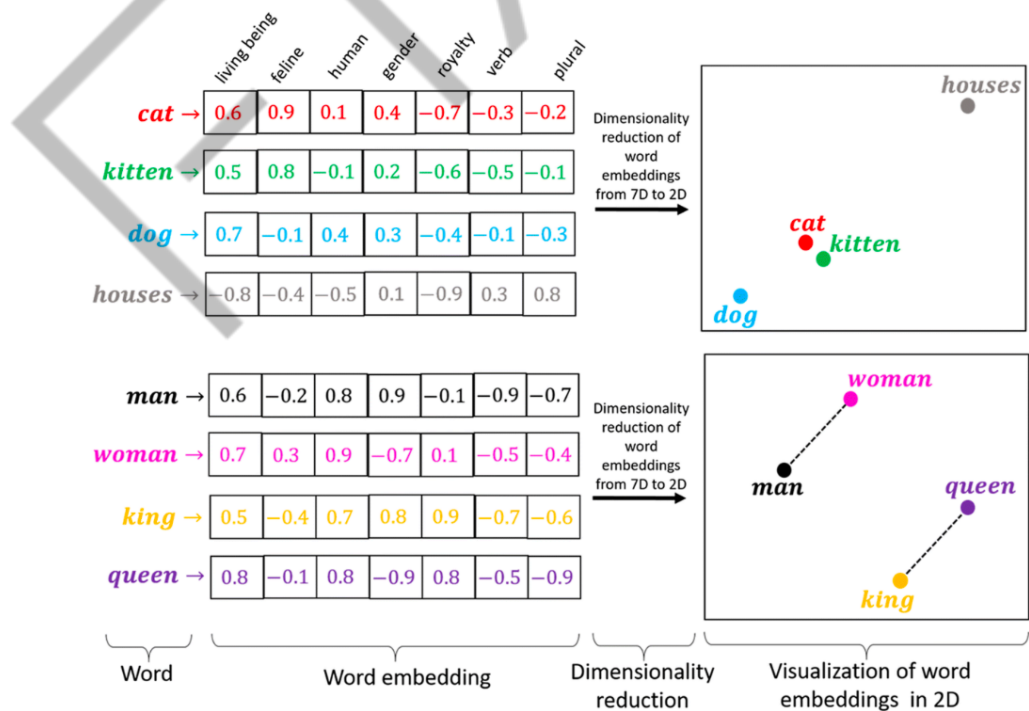


Figura 9 – Representação de textos como *embeddings*
Fonte: viso.ai (2024)

Os *embeddings* são criados por modelos de aprendizado de máquina treinados para capturar relações semânticas. Tradicionalmente, técnicas como *Word2Vec*, *GloVe* e *FastText* geravam vetores para palavras individuais com base em coocorrências de termos em grandes *corpora* de texto. No entanto, nas arquiteturas modernas, particularmente aquelas usadas em RAG, os *embeddings* geralmente são gerados para sentenças ou parágrafos inteiros, não apenas para palavras isoladas.

Modelos como *Sentence Transformers* e *LaBSE* estão entre os mais usados para essa finalidade, pois criam *embeddings* mais robustos que capturam o significado em níveis textuais mais amplos.

A geração de *embeddings* para texto completo permite que o sistema RAG identifique quais partes de uma base de conhecimento são mais relevantes para uma determinada consulta. Isso ocorre porque, após transformar tanto a consulta quanto os documentos em vetores de alta dimensão, é possível medir a similaridade semântica entre o vetor da consulta e os vetores dos documentos, selecionando os “mais próximos” como candidatos relevantes para fundamentar a resposta.

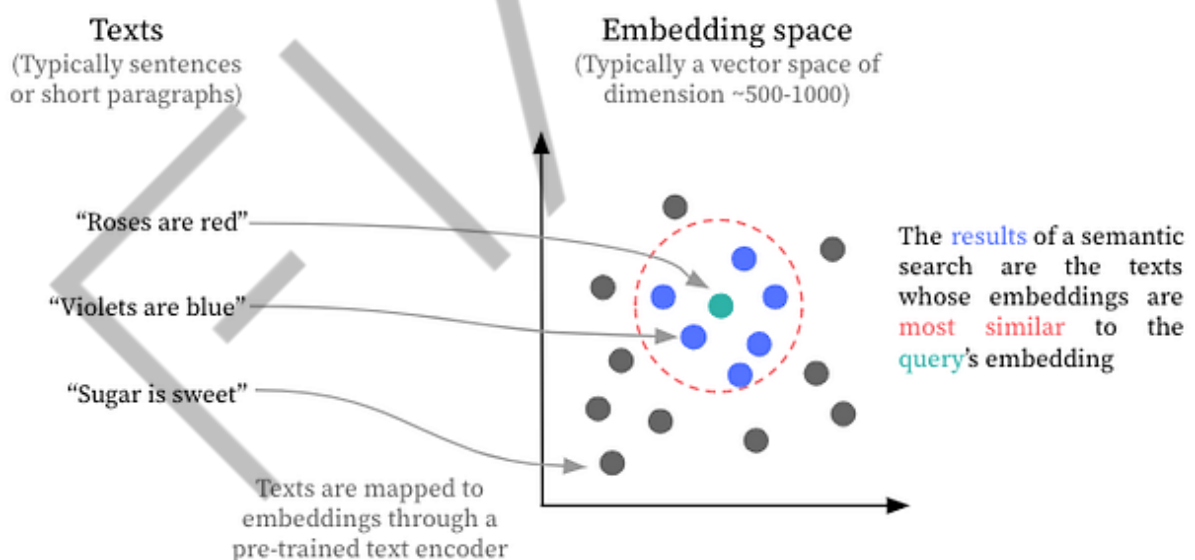


Figura 10 – Busca semântica
Fonte: viso.ai (2022)

Matematicamente, cada *embedding* é uma lista de valores numéricos que representa o significado do texto no espaço vetorial. A ideia é que partes de texto com significado semelhante fiquem próximas nesse espaço, o que facilita a comparação e recuperação. Essa noção está diretamente relacionada ao conceito de espaço latente

(*latent space*), um espaço de representação de dados em que relações semânticas emergem a partir da proximidade entre vetores.

No contexto do RAG, a etapa de recuperação funciona da seguinte forma: primeiro, cada segmento de texto da base de conhecimento é convertido em *embedding* e armazenado em um índice vetorial (geralmente em um banco de dados vetorial especializado). Quando o usuário faz uma pergunta, o texto da pergunta também é convertido em um *embedding*. Em seguida, realiza-se uma medida de similaridade entre o *embedding* da pergunta e os já armazenados para identificar os trechos de texto mais relevantes. Essa comparação pode ser feita por meio de métricas matemáticas como similaridade de cosseno ou distância euclidiana, que quantificam quão próximos dois vetores estão no espaço semântico.

Um exemplo didático dessa técnica com Python usando a biblioteca *Sentence Transformers* pode ser assim implementado:

```
1 | import numpy as np
2 | import matplotlib.pyplot as plt
3 |
4 | from sentence_transformers import SentenceTransformer,
  util
5 |
6 | # 1) EMBEDDINGS
7 | modelo = SentenceTransformer('all-MiniLM-L6-v2')
8 |
9 | documentos = [
10 |     "Este manual explica como instalar o software.",
11 |     "Guia de operação do sistema.",
12 |     "Especificações técnicas do produto."
13 | ]
14 |
15 | embeddings_docs = modelo.encode(documentos,
  convert_to_tensor=True)
16 | consulta = "Quais são os passos para instalar?"
17 | embedding_consulta = modelo.encode(consulta,
  convert_to_tensor=True)
18 |
19 | # Cosine similarities (query vs docs)
20 | similaridades = util.cos_sim(embedding_consulta,
  embeddings_docs).cpu().numpy().flatten()
21 |
22 | indice_mais_similar = int(np.argmax(similaridades))
23 | print("Documento mais relevante:",
  documentos[indice_mais_similar])
```

```
24| print("Similaridades (cos):", {i:
float(similaridades[i]) for i in range(len(documentos))})
25|
26| # 2) PCA (2D) MANUAL (SVD)
27| # Build matrix with query + docs
28| X = np.vstack([
29|     embedding_consulta.cpu().numpy(),
30|     embeddings_docs.cpu().numpy()
31| ]).astype(np.float64)
32|
33| labels = ["Consulta"] + [f"Doc {i}" for i in
range(len(documentos))]
34|
35| # Center
36| X_centered = X - X.mean(axis=0, keepdims=True)
37|
38| # SVD -> principal components
39| U, S, Vt = np.linalg.svd(X_centered,
full_matrices=False)
40| PC = Vt[:2].T # (d, 2)
41| X_2d = X_centered @ PC # (n, 2)
42|
43| # 3) DISTANCES IN PCA SPACE
44| q_xy = X_2d[0]
45| docs_xy = X_2d[1:]
46|
47| # Euclidean distances in PCA space (query -> each doc)
48| dist_pca = np.linalg.norm(docs_xy - q_xy, axis=1)
49|
50| # For reference: cosine distance in original space (1 -
cosine similarity)
51| dist_cos = 1.0 - similaridades
52|
53| print("Distâncias (PCA euclidiana) query->docs:", {i:
float(dist_pca[i]) for i in range(len(documentos))})
54| print("Distâncias (cos) query->docs:", {i:
float(dist_cos[i]) for i in range(len(documentos))})
55|
56| # 4) PLOTS
57| # Scatter plot in PCA space
58| plt.figure(figsize=(8, 6))
59| plt.scatter(X_2d[0, 0], X_2d[0, 1], marker="*", s=250,
label="Consulta")
60| for i in range(len(documentos)):
61|     plt.scatter(X_2d[i+1, 0], X_2d[i+1, 1], s=120,
label=f"Doc {i}")
62|     # line from query to doc
63|     plt.plot([X_2d[0, 0], X_2d[i+1, 0]], [X_2d[0, 1],
X_2d[i+1, 1]])
64|     # distance annotation
```

```

65|     midx = (X_2d[0, 0] + X_2d[i+1, 0]) / 2
66|     midy = (X_2d[0, 1] + X_2d[i+1, 1]) / 2
67|     plt.text(midx, midy, f"{dist_pca[i]:.3f}")
68|
69| plt.title("PCA (2D) – Consulta vs Documentos (linhas =
distâncias PCA)")
70| plt.xlabel("PC1")
71| plt.ylabel("PC2")
72| plt.legend()
73| plt.grid(True)
74| plt.show()
75|
76| # Bar chart of distances (PCA space)
77| plt.figure(figsize=(8, 4))
78| plt.bar([f"Doc {i}" for i in range(len(documentos))],
dist_pca)
79| plt.title("Distância Euclidiana no Espaço PCA (Consulta
→ Documento)")
80| plt.xlabel("Documento")
81| plt.ylabel("Distância (PCA)")
82| plt.grid(True, axis="y")
83| plt.show()
84|
85| # Bar chart of cosine distance (original embedding
space)
86| plt.figure(figsize=(8, 4))
87| plt.bar([f"Doc {i}" for i in range(len(documentos))],
dist_cos)
88| plt.title("Distância Cosseno no Espaço Original
(Consulta → Documento)")
89| plt.xlabel("Documento")
90| plt.ylabel("Distância (1 - cos_sim)")
91| plt.grid(True, axis="y")
92| plt.show()

```

Código-fonte 6: Compreendendo o processo de busca semântica
Fonte: Elaborado pelo autor (2026)

Nesse código, após carregar um modelo de criação de *embeddings*, transformamos tanto os documentos quanto a consulta em vetores. Em seguida, calculamos as similaridades entre o *embedding* da consulta e os dos documentos para identificar o trecho mais relevante. Esse processo ilustra como a busca semântica baseada em *embeddings* funciona como peça central no mecanismo de recuperação do RAG.

Em seguida, utilizamos o PCA para reduzir o tamanho do vetor de 384 para 2 dimensões. Assim, podemos fazer um gráfico e visualizar os *embeddings* dos textos posicionados no espaço euclidiano com 2 dimensões.

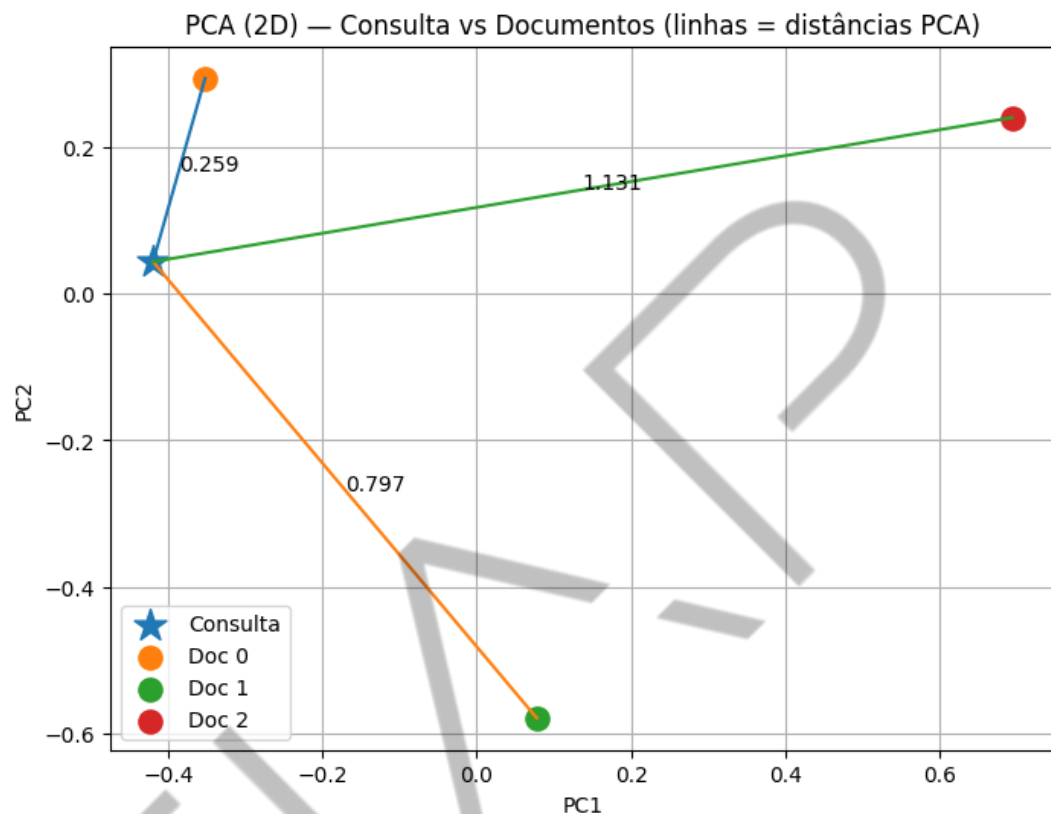


Figura 11 Visualizando a distância entre embeddings com 2 dimensões após uso de PCA
Fonte: Elaborado pelo autor (2026)

Além de recuperar documentos relevantes, *embeddings* também são usados em outras tarefas de NLP, como agrupamento de textos (*clustering*), recomendação de conteúdo e análise de similaridade semântica entre sentenças, o que reforça sua versatilidade em aplicações de inteligência artificial.

A escolha do modelo de *embeddings* impacta fortemente a qualidade da recuperação. Modelos como *text-embedding-3-large* da OpenAI oferecem *embeddings* de alta fidelidade que capturam nuances semânticas profundas, enquanto modelos menores podem oferecer bom desempenho com menor custo computacional.

Em síntese, os *embeddings* para recuperação são representações vetoriais densas que facilitam a comparação semântica de textos e permitem que sistemas

RAG identifiquem de forma eficaz os trechos de informação mais relevantes para responder uma consulta.

6 APLICAÇÕES DO RAG

Vamos explorar como a técnica de RAG vem sendo aplicada em contextos reais e variados, estendendo o uso de modelos de linguagem para cenários em que precisão, contexto específico e acesso a conhecimento atualizado são essenciais. A abordagem RAG combina recuperação de informação com geração de texto, superando limitações de modelos de linguagem puros que dependem apenas de seus dados de treinamento. Essa capacidade tem se mostrado útil em diferentes domínios industriais, acadêmicos e corporativos que exigem respostas fundamentadas em dados externos confiáveis. A técnica permite que sistemas de IA consultem e incorporem textos relevantes de bases de conhecimento externas antes de gerar respostas, o que resulta em maior acurácia, atualidade e contextualização das respostas em comparação com modelos tradicionais.

Uma das aplicações mais difundidas do RAG está no desenvolvimento de sistemas de atendimento ao cliente e chatbots inteligentes que conseguem responder a consultas específicas dos usuários com base em documentação interna, políticas de produto, históricos de interação ou FAQs corporativas. Em vez de oferecer respostas genéricas, um sistema RAG busca nos documentos da empresa informações relevantes e atualizadas antes de construir a resposta, o que melhora a experiência do cliente e reduz erros. Por exemplo, um assistente que responde a perguntas sobre políticas de devolução de uma loja pode recuperar trechos precisos dessas políticas e integrar esse contexto ao prompt que alimenta um LLM para gerar uma resposta clara e acurada.

Outra aplicação importante do RAG está na área jurídica e de conformidade legal. Advogados e profissionais jurídicos frequentemente precisam consultar grandes volumes de legislação, jurisprudências e contratos para responder a questões complexas ou preparar documentos legais. Com RAG, é possível desenvolver assistentes que recuperam precedentes legais pertinentes e trechos de normas específicas antes de gerar resumos ou sugestões. Isso economiza tempo e permite

respostas mais confiáveis em comparação com sistemas que dependem apenas de memória estática de um modelo de linguagem. Pesquisas também destacam aplicações acadêmicas e startups que usam RAG para auxiliar na pesquisa jurídica, integrando bases de leis e cases em fluxos automatizados de pergunta e resposta.

No setor de saúde e medicina, RAG tem sido utilizado para melhorar sistemas de suporte à decisão clínica e ferramentas de informação médica. Profissionais de saúde podem consultar rapidamente protocolos clínicos, guias terapêuticos ou artigos científicos relevantes em resposta a perguntas específicas sobre tratamentos ou diagnósticos. A integração de informações atualizadas e evidências revisadas por pares ajuda a reduzir riscos de decisões baseadas em dados incompletos ou desatualizados, sendo útil em contextos em que o conhecimento novo é constantemente produzido. Pesquisas acadêmicas recentes demonstram que *pipelines* RAG customizados para medicina conseguem alcançar níveis de precisão comparáveis a respostas humanas em tarefas de consulta sobre diretrizes pré-operatórias.

Empresas e organizações também aplicam RAG em sistemas de perguntas e respostas corporativos, em que funcionários precisam acessar rapidamente políticas internas, documentos técnicos, manuais de produto ou relatórios de projeto. Em vez de treinar modelos de linguagem em dados proprietários, que é um processo custoso e demorado, o uso de RAG permite que o sistema acesse essas fontes diretamente durante a inferência, mantendo as respostas atualizadas à medida que os documentos são atualizados. Isso é útil em grandes corporações que precisam democratizar acesso ao conhecimento interno de forma eficiente.

O uso de RAG também se estende a aplicações em educação e aprendizado personalizado. Plataformas educacionais podem utilizar RAG para responder perguntas baseadas em conteúdo didático específico, como livros digitais, apostilas ou materiais de curso. Um sistema RAG que recupera trechos relevantes das leituras obrigatórias antes de gerar uma resposta permite que os estudantes obtenham explicações vinculadas diretamente ao conteúdo curricular, o que pode facilitar a compreensão de tópicos complexos e promover um aprendizado mais contextualizado.

Além dessas áreas, diversos outros setores (como finanças, telecomunicações e análise de dados científicos) estão explorando RAG para resolver problemas de busca e síntese de informação em grandes volumes de dados. Em finanças, por exemplo, sistemas podem acessar relatórios de mercado e documentos regulatórios antes de gerar análises ou recomendações. Em telecomunicações, por sua vez, podem integrar logs de rede ou documentação técnica para responder a consultas de engenharia ou suporte. Esses casos ilustram como a combinação de recuperação e geração pode transformar fluxos de trabalho que antes exigiam pesquisa manual intensiva em processos automatizados e escaláveis.

Do ponto de vista prático de desenvolvimento, um pequeno sistema RAG para perguntas sobre um conjunto de documentos pode ser implementado em Python com bibliotecas como LangChain e um banco vetorial como ChromaDB ou FAISS. O seguinte exemplo demonstra como consultar um índice vetorial e gerar respostas contextualizadas usando um modelo de linguagem capaz de responder com base no contexto recuperado. Como base de conhecimento, utilizaremos o **manual do aluno da Fiap** para construirmos um sistema de suporte aos alunos:

```
1 | %pip install --quiet --upgrade langchain-text-splitters
  langchain-community langgraph langchain-openai langchain-
  core pypdf unstructured
2 |
3 | import getpass
4 | import os
5 | from google.colab import userdata
6 | from langchain.chat_models import init_chat_model
7 | from langchain_core.vectorstores import
  InMemoryVectorStore
8 | from langchain_openai import OpenAIEmbeddings
9 |
10|
11| os.environ["OPENAI_API_KEY"] =
  userdata.get('OPENAI_API_KEY')
12| llm = init_chat_model("gpt-4o-mini",
  model_provider="openai")
13| embeddings = OpenAIEmbeddings(model="text-embedding-ada-
  002")
14|
15| vector_store = InMemoryVectorStore(embeddings)
16|
17| #criando uma base de conhecimento
18| import bs4
```

```
19| from langchain_community.document_loaders import
PyPDFLoader
20|
21| file_path = "/content/manual_do_candidato_2026_ps2.pdf"
22| loader = PyPDFLoader(file_path)
23|
24| docs = loader.load()
25|
26| #fazendo o splitting dos documentos
27| from langchain_text_splitters import
RecursiveCharacterTextSplitter
28|
29| text_splitter = RecursiveCharacterTextSplitter(
30|     chunk_size=1000, # chunk size (characters)
31|     chunk_overlap=200, # chunk overlap (characters)
32|     add_start_index=True, # track index in original
document
33| )
34| all_splits = text_splitter.split_documents(docs)
35|
36| print(f"Split pdf into {len(all_splits)} sub-
documents.")
37|
38| #guardando os dados em um banco de dados
39| document_ids =
vector_store.add_documents(documents=all_splits)
40|
41| # Create a LANGSMITH_API_KEY in Settings > API Keys
42| from langsmith import Client
43| client =
Client(api_key=userdata.get('LANGSMITH_API_KEY'))
44| prompt = client.pull_prompt("rlm/rag-prompt",
include_model=True)
45|
46| prompt.messages[0].prompt.template = '''You are an
assistant for question-answering tasks. Use the following
pieces of retrieved context to answer the question. If you
don't know the answer, just say 'NAO SEI'. Use three
sentences maximum and keep the answer concise.
47| Question: {question}
48| Context: {context}
49| Answer:'''
50|
51| from langchain_core.documents import Document
52| from typing_extensions import List, TypedDict
53|
54|
55| class State(TypedDict):
56|     question: str
57|     context: List[Document]
```



```
58|     answer: str
59|
60| def retrieve(state: State):
61|     retrieved_docs =
vector_store.similarity_search(state["question"])
62|     return {"context": retrieved_docs}
63|
64|
65| def generate(state: State):
66|     docs_content = "\n\n".join(doc.page_content for doc
in state["context"])
67|     messages = prompt.invoke({"question":
state["question"], "context": docs_content})
68|     response = llm.invoke(messages)
69|     return {"answer": response.content}
70|
71| #definindo o workflow
72| from langgraph.graph import START, StateGraph
73|
74| graph_builder =
StateGraph(State).add_sequence([retrieve, generate])
75| graph_builder.add_edge(START, "retrieve")
76| graph = graph_builder.compile()
77|
78| result = graph.invoke({"question": "qual é o valor da
taxa de inscrição?"})
79|
80| print(f'Context: {result["context"]}\n\n')
81| print(f'Answer: {result["answer"]}')

```

Código-fonte 7: Sistema RAG para responder a dúvidas sobre o processo seletivo da FIAP
Fonte: Elaborado pelo autor (2026)

Nesse exemplo simplificado, os documentos são convertidos em vetores e indexados. A consulta é transformada em *embedding* e utilizada para recuperar trechos relevantes, que posteriormente fornecem contexto ao modelo de linguagem antes da geração da resposta. Essa integração mostra como sistemas RAG podem ser adaptados para aplicações práticas em empresas e serviços que dependem de conhecimento específico e atualizado.

Em resumo, as aplicações do RAG abrangem um amplo espectro de casos de uso que exigem respostas com conhecimento atualizado, acessível e específico de domínio. Ao recuperar informações relevantes de bases de conhecimento externas antes de gerar respostas, sistemas RAG conseguem superar limitações de modelos de linguagem tradicionais e oferecer soluções que combinam precisão,

contextualização e atualidade em uma variedade de ambientes industriais e acadêmicos.

7 DESAFIOS

Vamos entender de forma aprofundada os desafios e limitações enfrentados por sistemas baseados em *Retrieval-Augmented Generation*. Embora RAG represente uma evolução importante, essa técnica não elimina todos os problemas associados à inteligência artificial generativa. Existem desafios técnicos, arquiteturais e de avaliação que precisam ser compreendidos por quem projeta ou implementa sistemas reais com RAG.

Um dos principais desafios é que RAG não evita completamente as alucinações por parte dos modelos de linguagem grandes, apesar de reduzir sua frequência. Mesmo quando um sistema RAG recupera trechos de documentos corretos e factuais, o modelo gerador pode ainda assim produzir respostas que extrapolam ou distorcem essas informações se interpretar mal o contexto. Isso ocorre porque a geração de texto continua sendo um processo probabilístico do modelo de linguagem, que pode “chutar” respostas plausíveis a partir do contexto fornecido. Por exemplo, um modelo pode combinar partes de fontes distintas de maneira imprecisa e fornecer uma conclusão incorreta, mesmo que a base de dados contenha informação correta. Esse fenômeno está documentado em estudos que mostram que métodos de detecção de alucinações baseados em similaridade semântica muitas vezes falham em contextos reais, indicando que sistemas RAG ainda não são completamente à prova de alucinações por si só.

Outro desafio significativo é a eficiência da recuperação e a relevância do conteúdo selecionado. A eficácia de um sistema RAG depende diretamente da qualidade do mecanismo de *retrieval*. Se os *embeddings* gerados para a consulta ou para os documentos não representarem adequadamente o significado semântico, o sistema pode recuperar trechos pouco relevantes, o que impacta negativamente a qualidade da resposta final. A literatura técnica destaca que métricas de eficiência de recuperação, como precisão e *recall* para os documentos mais relevantes (*Precision@k*, *Recall@k*), são cruciais para avaliar o desempenho de sistemas RAG,

mas ainda faltam práticas consolidadas de *benchmark* e avaliação comparativa em diferentes domínios e arquiteturas.

Um aspecto relacionado é o custo computacional e de latência. O uso de RAG normalmente envolve múltiplas etapas que precisam ser executadas em sequência: criação de *embeddings*, consulta ao banco de dados vetorial, recuperação de trechos relevantes e geração de resposta. Cada uma dessas etapas consome recursos e tempo e, quando combinadas, podem resultar em latência perceptível no tempo de resposta, especialmente em aplicações em tempo real ou sistemas de grande escala.

A necessidade de acessar um banco de dados vetorial e de processar múltiplas chamadas ao modelo de linguagem acrescenta complexidade em relação a um LLM operando de forma isolada. Isso significa que implementações de RAG em produção precisam focar em otimização de performance e infraestrutura robusta para garantir que o sistema responda de forma responsiva.

A escolha e o balanceamento dos parâmetros de recuperação também representam um desafio prático. Decidir quantos documentos recuperar (*top-k*), qual métrica de similaridade usar (cosseno ou distância euclidiana) e como reordenar os resultados (*re-ranking*) pode depender do domínio de aplicação e do tamanho da base de conhecimento. Ajustes ineficazes podem tanto reduzir a precisão quanto aumentar a sobrecarga computacional. Pesquisas sugerem que abordagens híbridas de recuperação, combinando métodos baseados em palavras-chave com métodos densos de *embeddings*, podem melhorar a relevância e reduzir taxas de alucinação, mas isso adiciona outro nível de complexidade ao design do sistema.

A avaliação de sistemas RAG também apresenta dificuldades únicas. Ao contrário de modelos de linguagem isolados, que podem ser avaliados por métricas padronizadas de geração de texto ou coerência, um sistema RAG envolve tanto a recuperação de informação quanto a geração de texto contextualizada. Isso requer métricas que avaliem não apenas a qualidade da resposta final, mas também a relevância e precisão dos documentos recuperados e a consistência entre a evidência recuperada e a resposta produzida. A pesquisa acadêmica destaca a necessidade de abordagens de avaliação que integrem *benchmarks* específicos de tarefas, avaliações humanas e métricas que considerem a ligação entre recuperação e geração.

Outro desafio, frequentemente negligenciado, é realizar atualizações contínuas na base de conhecimento externa. Embora um dos benefícios do RAG seja a capacidade de atualizar a base de dados sem retreinar o modelo *generator*, a integração incremental de novos dados exige processos de ingestão e reindexação eficientes, além de mecanismos para lidar com redundâncias ou dados conflitantes.

Uma base de conhecimento mal organizada ou desatualizada pode degradar a performance do sistema ao recuperar trechos irrelevantes ou desatualizados, o que contrapõe ao objetivo de manter respostas precisas e atualizadas.

A gestão de dados contraditórios ou conflitantes é outro ponto crítico. Sistemas RAG podem recuperar fontes que, embora factualmente corretas individualmente, contenham informações conflitantes sobre um tema. Nesses casos, o modelo pode misturar fragmentos de distintas fontes e produzir uma resposta que abrange dados antigos e novos de forma enganosa, dificultando a confiabilidade da saída final. Esse problema é especialmente relevante em domínios como regulamentação legal ou protocolos clínicos em constante atualização, em que a ordem cronológica e a precisão contextual são cruciais.

Por fim, aspectos de segurança, privacidade e conformidade também são desafios em sistemas RAG, sobretudo quando lidam com dados sensíveis ou proprietários. A centralização de conteúdo em bancos de dados vetoriais pode exigir cuidados com controle de acesso e criptografia para garantir que informações confidenciais não sejam expostas indevidamente durante consultas ou na integração com modelos de geração. Esses requisitos tornam a implementação de RAG em contextos regulamentados mais complexa, exigindo políticas de governança de dados bem definidas.

Em síntese, embora a técnica de RAG ofereça uma maneira poderosa de mitigar limitações de LLMs e integrar conhecimento externo de forma eficaz, o seu uso real implica enfrentar desafios práticos e teóricos que vão muito além da simples integração de recuperação e geração. A pesquisa acadêmica e as práticas na indústria continuam evoluindo para desenvolver métricas de avaliação mais robustas, estratégias de recuperação mais eficientes e abordagens que reduzem custos de computação e melhorem a segurança e a precisão das respostas.

8 CONCLUSÃO

Ao longo deste capítulo, exploramos em profundidade o conceito de *Retrieval-Augmented Generation*, uma técnica que une a geração de linguagem natural por grandes modelos com mecanismos de recuperação de informação externa para produzir respostas mais precisas, atualizadas e contextualizadas do que seria possível com modelos de linguagem isolados. Iniciamos examinando as limitações intrínsecas dos LLMs, como conhecimento estático, capacidade de alucinação e janelas de contexto finitas, e demonstramos como essas limitações motivaram a adoção de arquiteturas híbridas como RAG, que permitem *grounding* das respostas em bases de conhecimento externas acessíveis em tempo real.

O fluxo de um sistema RAG envolve a conversão da consulta do usuário em uma representação vetorial, a busca por trechos relevantes em um banco de dados vetorial, a combinação desses trechos ao prompt e, finalmente, a geração de uma resposta fundamentada por um LLM. Essa combinação permite que sistemas RAG lidem com tarefas intensivas em conhecimento, como respostas a perguntas específicas de domínio, resumo de documentos ou atendimento automático em cenários corporativos, beneficiando-se de informações que não estavam presentes nos dados originais de treinamento do modelo.

Vimos também que os componentes principais de um sistema RAG, a base de conhecimento, *embeddings* semânticos, mecanismos de recuperação e modelos geradores, funcionam de forma integrada para tornar possível essa recuperação de dados em tempo real, otimizando a qualidade das respostas. A geração de *embeddings* e a busca por similaridade semântica foram apresentadas como fundamentos matemáticos que permitem comparar textos e identificar aqueles mais relevantes para uma consulta específica.

As aplicações de RAG demonstram seu impacto prático em setores como atendimento ao cliente, jurídico, saúde, pesquisa acadêmica e gestão de conhecimento. Em cada um desses contextos, RAG oferece uma forma eficiente de acessar informação atualizada e específica de domínio, reduzindo custos de *retraining* e melhorando a precisão das interações de IA.

No entanto, também discutimos desafios e limitações que acompanham essa tecnologia, incluindo a persistência de alucinações, a necessidade de recuperação relevante e eficiente, custos de computação e problemas de avaliação de sistemas integrados. Tópicos como atualização contínua da base de conhecimento e a necessidade de métricas robustas para medir a qualidade tanto da recuperação quanto da resposta gerada foram destacados como áreas de pesquisa ativa.

Por fim, RAG continua evoluindo como campo de pesquisa, com propostas que buscam melhorar o raciocínio multi-etapa, sucesso em recuperação híbrida, *re-ranking* de resultados e integração de dados heterogêneos ou multimodais. Estudos recentes também exploram como essas técnicas podem ser aplicadas em contextos especializados, como medicina e engenharia de software, apontando para uma rica agenda de trabalho futuro.

Em conjunto, o material fornece não apenas a base conceitual e técnica para entender RAG, mas também um panorama amplo de seus usos, benefícios e limitações, preparando o leitor para aplicar essas técnicas em contextos reais ou aprofundar-se em pesquisas avançadas sobre sistemas generativos com recuperação de informação.

REFERÊNCIAS

CHROMADB DOCUMENTATION. *ChromaDB Documentation*. Disponível em: <https://docs.trychroma.com/docs/overview/getting-started>. Acesso em: 22 mai. 2026.

GOODFELLOW, Ian J. *et al.* *Generative adversarial nets*. In: *Advances in Neural Information Processing Systems* 27. [S.l.]: NeurIPS, 2014.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep learning*. Cambridge: MIT Press, 2016. Disponível em: <http://www.deeplearningbook.org>. Acesso em: 22 mai. 2026.

HOFFMANN, Jordan *et al.* *Training compute-optimal large language models*. arXiv preprint arXiv:2203.15556, 2022.

LANGCHAIN DOCUMENTATION. *LangChain Documentation*. Disponível em: <https://docs.langchain.com/>. Acesso em: 22 mai 2026.

OPENAI COOKBOOK. *OpenAI Cookbook*. Disponível em: <https://developers.openai.com/cookbook>. Acesso em: 22 mai. 2026.

VASWANI, Ashish *et al.* *Attention is all you need*. In: *Advances in Neural Information Processing Systems* 30. [S.l.]: NeurIPS, 2017.

YENDURI, G. *et al.* *GPT (Generative Pre-Trained Transformer) — A Comprehensive Review on Enabling Technologies, Potential Applications, Emerging Challenges, and Future Directions*. *IEEE Access*, v. 12, p. 54608–54649, 2024.